



מבוא למדעי המחשב מ"ח' (234114 \ 234117)

סמסטר חורף תשע"ט

מבחן מסכם מועד ב', 14 במרץ 2019

2	3	4	1	1	
---	---	---	---	---	--

רשום/ה לקורס:

--	--	--	--	--	--	--	--	--	--

מספר סטודנט:

משך המבחן: 3 שעות.

חומר עזר: אין להשתמש בכל חומר עזר.

הנחיות כלליות:

- מלאו את הפרטים בראש דף זה ובדף השער המצורף, בעט בלבד.
- בדקו שיש 22 עמודים (4 שאלות) במבחן, כולל עמוד זה.
- כתבו את התשובות על טופס המבחן בלבד, במקומות המיועדים לכך. שימו לב שהמקום המיועד לתשובה אינו מעיד בהכרח על אורך התשובה הנכונה.
- העמודים הזוגיים בבחינה ריקים. ניתן להשתמש בהם כדפי טיוטה וכן לכתיבת תשובותיכם. סמנו טיוטות באופן ברור על מנת שהן לא תיבדקנה.
- יש לכתוב באופן ברור, נקי ומסודר, ובעט בלבד.
- בכל השאלות, הנכם רשאים להגדיר ולממש פונקציות עזר כרצונכם. לנוחיותכם, אין חשיבות לסדר מימוש הפונקציות בשאלה, ובפרט ניתן לממש פונקציה לאחר השימוש בה.
- אלא אם כן נאמר אחרת בשאלות, אין להשתמש בפונקציות ספריה או בפונקציות שמומשו בכיתה, למעט פונקציות קלט/פלט והקצאת זיכרון (`malloc`, `free`). ניתן להשתמש בטיפוס `bool` המוגדר ב-`stdbool.h`.
- אין להשתמש במשתנים סטטיים וגלובאליים אלא אם נדרשתם לכך מפורשות.
- כשאתם נדרשים לכתוב קוד באילוצי סיבוכיות זמן/מקום נתונים, אם לא תעמדו באילוצים אלה תוכלו לקבל בחזרה מקצת הנקודות אם תחשבו נכון ותציינו את הסיבוכיות שהצלחתם להשיג.
- נוהל "לא יודע": אם תכתבו בצורה ברורה "לא יודע/ת" על שאלה (או סעיף) שבה אתם נדרשים לקודד, תקבלו 20% מהניקוד. דבר זה מומלץ אם אתם יודעים שאתם לא יודעים את התשובה.
- נוסחאות שימושיות:

$$1 + \frac{1}{2} + \frac{1}{3} + \dots + \frac{1}{n} = \Theta(\log n) \quad 1 + \frac{1}{4} + \frac{1}{9} + \frac{1}{16} + \frac{1}{25} + \dots = \Theta(1)$$

$$1^k + 2^k + 3^k + \dots + n^k = \Theta(n^{k+1}) \quad 1 + k + k^2 + k^3 + \dots + k^n = \Theta(k^n)$$

צוות הקורס 234114/7

מרצים: פרופ' תומר שלומי, ד"ר אהרון קופרשטוק, ד"ר רמי כהן, מר טל טבצ'ניק **מתרגלים:** דניאלה בר לב, עדי גרוס, עמר דהרי, ניר חכמוביץ, יארה מולא, גסוב מזאוי, נגיב נבואני, דניאל עוז, אלון פאר, דמיטרי רבינוביץ' (מתרגל אחראי), יאיר ריעאני, עידו רפאל, אסף שומר

בהצלחה!

[illegible]



שאלה 1 (25 נקודות):

א. (8 נקודות) חשבו את סיבוכיות הזמן והמקום של הפונקציה $f1$ המוגדרת בקטע הקוד הבא, כפונקציה של n , כאשר n הוא אורך המחרוזת str . אין צורך לפרט שיקולים. חובה לפשט את הביטוי ככל שניתן.

```
int g(char *str)
{
    int length = 0;
    while (*(str + length))
        ++length;
    return length;
}

void f1(char *str)
{
    for (int i = 0; i < g(str); ++i)
        if (str[i] >= 'a' && str[i] <= 'z')
            str[i] += 'A' - 'a';
}
```

סיבוכיות זמן: $\Theta(\text{_____})$ סיבוכיות מקום: $\Theta(\text{_____})$

ב. (9 נקודות) חשבו את סיבוכיות הזמן והמקום של הפונקציה $f2$ המוגדרת בקטע הקוד הבא, כפונקציה של n . אין צורך לפרט שיקולים. חובה לפשט את הביטוי ככל שניתן.

```
int g(int n, int k)
{
    if (n <= k) return 1;

    int result = 0;
    for (int i = k; i > 0; --i, ++result);

    return result + g(n - 1, 2 * k);
}

int f2(int n)
{
    return g(n, 2);
}
```

סיבוכיות זמן: $\Theta(\text{_____})$ סיבוכיות מקום: $\Theta(\text{_____})$

[illegible]



ג. (8 נקודות): חשבו את סיבוכיות הזמן והמקום של הפונקציה f3 המוגדרת בקטע הקוד הבא, כפונקציה של n. אין צורך לפרט שיקולים. חובה לפשט את הביטוי ככל שניתן.

```
int g(int n)
{
    if (n <= 1) return 1;
    return 8 * g(n / 2);
}

void f3(int n)
{
    int k = g(n);

    for (int i = 2; i < k * k; ++i)
    {
        printf("*");
    }
}
```

סיבוכיות זמן: $\Theta(\quad)$ סיבוכיות מקום: $\Theta(\quad)$

[illegible]



שאלה 2 (25 נק')

נתונים שני מערכים ממוינים של מספרים שלמים. מערך a באורך na הממוין בסדר לא עולה ומערך b באורך nb הממוין בסדר לא יורד. ממשו פונקציה

```
int meeting_point(int a[], int na, int b[], int nb)
```

שתחזיר את האינדקס i הקטן ביותר שמקיים $a[i] == b[i]$, או -1 אם לא קיים אינדקס כזה.

לדוגמה: אם נתונים מערכים הבאים

$a[]$	=	19	19	16	15	15	15	15	13	-2
--------	---	----	----	----	----	----	----	----	----	----

$b[]$	=	0	12	13	14	14	15	15	19	42	57
--------	---	---	----	----	----	----	----	----	----	----	----

עבור קריאה `meeting_point(a, 9, b, 10)`, הפונקציה תחזיר 5

עבור קריאה `meeting_point(a, 9, b, 5)`, הפונקציה תחזיר -1

עבור קריאה `meeting_point(a, 3, b, 8)`, הפונקציה תחזיר -1

דרישות:

- סיבוכיות זמן: $O(\log n)$.

אם לפי חישוביכם לא עמדתם בדרישות הסיבוכיות אנא ציינו כאן את הסיבוכיות שהגעתם אליה:

זמן _____ מקום נוסף _____

<pre>int meeting_point(int a[], int na, int b[], int nb)</pre>
<pre>{</pre>

[illegible]

[illegible]



שאלה 3 (25 נקודות) :

הגדרה: מחרוזת תקרא **מדהימה** אם כל אות שמופיע בה אינה חוזרת יותר מפעם אחת.

לדוגמה,

abcd, is, a, special, word כולן מדהימות.

בעוד, stunning, well, nooooot לא.

ממשו פונקציה שמקבלת מחרוזת `str` ומחזירה את האורך של תת-מחרוזת מדהימה ארוכה ביותר של `str`.

לדוגמה,

עבור $str = abcaacb$ הפונקציה תחזיר 3 מכיוון שתת-המחרוזות abc , bca , acb הן מדהימות ובאורך 3 ואין תת-מחרוזת מדהימה ארוכה יותר.

עבור $str = abcdef$ הפונקציה תחזיר 6, מכיוון שהמחרוזת עצמה מדהימה.

עבור $str = aaaaaa$ הפונקציה תחזיר 1, מכיוון שתת-המחרוזת המדהימה הארוכה ביותר היא a .

דרישות:

- סיבוכיות זמן $\Theta(n)$ וסיבוכיות מקום נוסף $\Theta(1)$, כאשר n הוא אורך המחרוזת.

אם לפי חישוביכם לא עמדתם בדרישות הסיבוכיות אנא ציינו כאן את הסיבוכיות שהגעתם אליה:

זמן _____ מקום נוסף _____

<code>int length_of_longest_stunning_substr(char *str) {</code>

[illegible]

[illegible]

[illegible]



שאלה 4 (25 נקודות) :

השאלה עוסקת בהכללה של בעיית סודוקו מגודל סריג של $3^2 \times 3^2$ לגודל סריג של $N^2 \times N^2$.

בהינתן סריג ריבועי בגודל $N^2 \times N^2$ עם מספרים בתחום בין 1 ל- N^2 בחלק מהתאים, המטרה בסודוקו היא להציב מספרים באותו התחום בכל אחד מהתאים הנותרים, כך שבכל עמודה, בכל שורה, ובכל ריבוע לא חופף בגודל $N \times N$ (כפי שמסומן בקווים קהים בציור שלמטה) ימצאו כל המספרים שבין 1 לבין N^2 .

לדוגמה: סריג נתון בגודל $3^2 \times 3^2$ למטה מימין, ופתרון לבעיה למטה משמאל..

5	3	4	6	7	8	9	1	2
6	7	2	1	9	5	3	4	8
1	9	8	3	4	2	5	6	7
8	5	9	7	6	1	4	2	3
4	2	6	8	5	3	7	9	1
7	1	3	9	2	4	8	5	6
9	6	1	5	3	7	2	8	4
2	8	7	4	1	9	6	3	5
3	4	5	2	8	6	1	7	9

5	3			7				
6			1	9	5			
	9	8					6	
8				6				3
4			8		3			1
7				2				6
	6					2	8	
			4	1	9			5
				8			7	9

ממשו פונקציה אשר מקבלת סריג המיוצג על ידי מערך דו-ממדי $N^2 \times N^2$ (עם מספר בין 1 ל- N^2 בכל מקום, או הערך 0 עבור מקומות בהם יש להציב מספר) ומחשבת את מספר הפתרונות האפשריים. אם אין אף פתרון שמקיים את האילוצים המתוארים למעלה, הפונקציה תחזיר 0.

```
int count_solutions(int grid[N*N][N*N])
```

הערות:

- יש להשתמש בשיטת backtracking כפי שנלמדה בכיתה.
- בשאלה זו אין דרישות סיבוכיות, אולם כמקובל ב-backtracking יש לוודא שלא מתבצעות קריאות רקורסיביות מיותרות עם פתרונות שאינם חוקיים.
- ניתן ומומלץ להשתמש בפונקציות עזר (ויש לממש את כולן).

```
int count_solutions(int grid[N*N][N*N])
```

```
{
```

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]